Promo Finder.

Code owns the clock. The LLM owns the drama.

Aryan Mehra
AI Generalist, AI Lab
github.com/iaryan08/kuku-promo-finder
promocraft-ai.streamlit.app

01 Why Problem 1

Editors scrub 50 to 100 episodes by hand to find promo clips, and new shows land every week. That is a real bottleneck, and promos are where growth starts. The pack had 50 timestamped screenplays of *I Married a Broken Soldier. He was a Duke*, so I could test on real data.

Skipped 2: a wrong statute is a liability and I cannot verify a legal corpus in hours. **Skipped 3:** no retention data to anchor it.

02 Assumptions (the brief was silent)

- **Window:** 20 to 60 s. Under 20 s there is no story. Past 60 s, retention drops.
- **Hook:** conflict, betrayal or a shock line in the first 3 s.
- **Cliffhanger:** cut at peak tension, before anyone resolves it.
- **Standalone:** the viewer knows nothing. It has to land cold.
- **Scope:** I find moments and cut points only. No editing, music or overlays.
- **Human:** an editor makes the final pick.

03 Technical Architecture



STAGE	OWNER	JOB
1 Parser `parser.py`	Regex	Reads scenes, dialogue, `ACTION` and `CAMERA` cues. Turns `MM:SS.ss` into float seconds. Raw 528,811 chars become 381,362 (28% less).
2 Windows `scorer.py`	Code	Slides across events, not whole scenes. Builds 565 candidate windows of 20 to 60 s across all 50 episodes.
3 Evaluator	**Gemini Flash, JSON mode**	Scores Hook, Cliffhanger, Standalone (0 to 10). Picks the exact sub-window, writes one line why. Trims under 20 s revert to the full window. Cascade: `3.8-flash`, `3.6-flash`, `2.5-flash`, `3.5-flash`, `pro-latest`, then the local engine.
4 Ranker `run.py`	Code	Sums to /30. Dedups in 10 s buckets. Keeps 16 non-overlapping promos in `promos.json`.
5 Offline mode	Drama Heuristic Engine	Conflict keyword density, dialogue volatility, speaker diversity. No network. Powers `--confidential`. Used for this run.

WHY NOT JUST PROMPT AN LLM ON THE RAW SCRIPTS?

"No. If you just prompt an LLM on raw scripts, it hallucinates timestamps, wastes tokens on routine dialogue, and fails on single-scene episodes like Episode 8. We used deterministic Python code for the math, timecode boundaries, sliding windows, and 28% token pre-filtering. We reserved the LLM strictly for what code cannot do: assessing human dramatic tension in the first 3 seconds and evaluating cliffhanger curiosity."

04 Unit Economics and Scale

METRIC	PROTOTYPE (THIS SUBMISSION)	PRODUCTION (10 SHOWS / WEEK)
Coverage	Heuristic engine scores all 565 windows	LLM scores all 565 (~11 per episode)
Model	On-device Drama Heuristic Engine	Gemini 3.8 Flash, paid Batch API
Tokens / show	None. Zero API calls.	~220K in, ~25K out
Cost / show	**$0.00**	**~$0.13** 220K x $0.375/M + 25K x $1.875/M (50% batch)
Run time	~2 s, fully local	~45 s (async batch pool)
Output goes to	CLI and Streamlit review	Slack webhook, Premiere markers

What changes for the editor: open the top clips, check cuts, export. About 2 hours becomes **under 5 minutes** per show, $1.30 a week for 10 shows.

05 Top Discovered Promos

#	EP	CLIP (MM:SS.ss)	LEN	SCORE /30
1	EP13	00:45.80-01:34.00	48.2s	28
2	EP43	00:43.90-01:33.20	49.3s	26
3	EP01	00:00.00-00:32.80	32.8s	25
4	EP30	00:58.80-01:49.00	50.2s	25
5	EP32	00:58.00-01:47.20	49.2s	25
6	EP36	00:00.00-00:48.80	48.8s	25
7	EP39	00:13.80-00:50.00	36.2s	25
8	EP09	00:28.80-01:02.00	33.2s	24
9	EP21	00:54.10-01:54.00	59.9s	24
10	EP25	00:44.40-01:30.20	45.8s	24

#1 EP 13 | H10 C9 S9 — 28/30

Public confrontation, gasping onlookers, open mockery. Cuts on an unresolved power struggle: the antagonist declares a verdict before Juliana can answer. **Why first:** maximum social stakes, ends before it resolves.

#2 EP 43 | H9 C9 S8 — 26/30

The antagonist threatens to burn a sentimental heirloom. Cuts on "how dare you", a question the clip refuses to answer.

#3 EP 01 | H9 C8 S8 — 25/30

Juliana at the altar, the wedding betrayal trope. Cuts mid-standoff before she responds. Episode 1 is an entry point by design.

Scoring note. This run used the local heuristic engine, not an LLM, because free-tier quota ran out. 565 windows built, 16 promos kept. **LLM cross-check:** I scored 60 windows (top 40 heuristic plus 20 random) with gemini-3.8-flash. Top-10 overlap was 3 of 10 and Spearman 0.12, so the engines mostly disagree. I have not hand-labelled to say which is right. LLM top 3: Ep 41 (00:00.00-00:48.50), Ep 16 (00:14.30-01:03.00), Ep 01 (00:29.80-01:19.00). Data: `raw_outputs/llm_comparison_results.json`.

06 Where It Picks Badly

Action with no dialogue — Episode 18: a cottage burns and Alexander breaks through the door. The drama sits in `ACTION` and `CAMERA` blocks. The only line is "Fire". The scorer rated it low on standalone. FIX A pre-LLM scan that boosts windows with verbs like slaps, tackles, draws sword, collapses.

Cross-episode arcs — Episode N often ends on a cliffhanger that pays off in the first seconds of N+1. My windows stay inside one episode. The brief allows multi-episode promos, so this is a real gap. FIX Stitch the last 20 s of N to the first 20 s of N+1.

Polite menace — Veiled aristocratic threats score below shouting. FIX Few-shot examples of quiet tension.

Keyword proxies — This run's scores are keyword proxies, not judgment. They can reward noise over tension. The 60-window LLM check disagreed with them (Spearman 0.12). Which one is right needs hand-labelled windows.

07 Re-Run, Adoption and Confidential Mode

```
$ python run.py --input /path/to/show_pack.zip
$ python -m streamlit run app.py
$ python run.py --confidential  # 100% on-device, zero cloud API calls
```

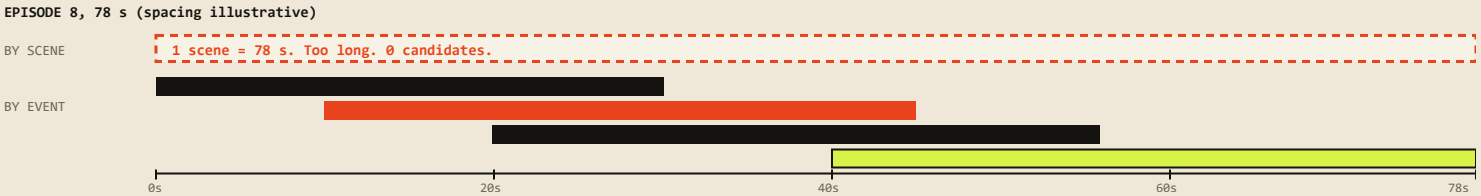
CONFIDENTIAL MODE Passing `--confidential` or flipping the UI toggle runs everything on-device through the Drama Heuristic Engine. **Zero cloud API calls.** Unreleased screenplays never leave the machine.

- Screenplays and API keys are git-ignored. Streamlit opens the 16 cached cuts without a key (live at **promocraft-ai.streamlit.app**).
- Swapping in a stronger model means changing one config value. The fallback chain stays.
- **How we would know it works:** the share of top clips an editor keeps unchanged, and click-through of tool-picked promos against hand-picked ones.

08 AI Tools and Where They Got It Wrong

Used: Claude Sonnet 4.6 (Thinking) for architecture reasoning, the regex parser and the UI. Gemini 3.8 Flash as the primary scorer and for the 60-window cross-check.

Failure, Episode 8. Claude assumed scenes are evenly sized and told me to cut candidates on scene boundaries. Episode 8 is one unsegmented 78 s scene. That design produced **zero** 20 to 60 s windows. I threw the design out and wrote event-level sliding windows in plain Python.



Lesson: the model never touches timecodes or window math. Code does.